

Chapitre 1 : Représentation Numérique

Binaire, Hexadécimal, Entiers et Flottants

Dr. El Hadji Bassirou TOURÉ
Département de Mathématiques et Informatique
Faculté des Sciences et Techniques
Université Cheikh Anta Diop de Dakar

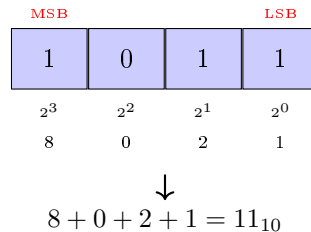
1 Systèmes de Numération

1.1 Base Binaire

Notation binaire

En base 2, chaque position représente une puissance de 2. Un nombre binaire $b_{n-1}b_{n-2}...b_1b_0$ correspond à $\sum_{i=0}^{n-1} b_i \cdot 2^i$ où $b_i \in \{0,1\}$. Le bit de poids fort (MSB) est à gauche, le bit de poids faible (LSB) à droite.

$$1011_2 = ?_{10}$$



Conversion décimal → binaire

Division successive par 2, en conservant les restes. Lire les restes de bas en haut (dernier reste = MSB).

$13_{10} \rightarrow$ **binaire**

$13 \div 2 = 6$ reste 1 (LSB)
$6 \div 2 = 3$ reste 0
$3 \div 2 = 1$ reste 1
$1 \div 2 = 0$ reste 1 (MSB)

$\uparrow$ **1101₂**

1.2 Base Hexadécimale

Notation hexadécimale

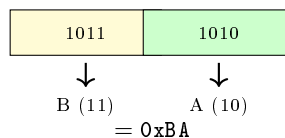
Base 16 utilise les symboles 0-9 et A-F (A=10, B=11, C=12, D=13, E=14, F=15). Préfixe 0x en C. Chaque chiffre hex = 4 bits exactement.

Hex	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Dec	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Bin	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111

Conversion binaire ↔ hexadécimal

Grouper par 4 bits depuis la droite. Chaque groupe = 1 chiffre hex. Ajouter des zéros à gauche si nécessaire.

$$10111010_2 \rightarrow \text{hex}$$



Listing 1: Notations en C

```

1 unsigned int x = 0x3A;    // 3A hex = 0011 1010 bin = 58 dec
2 unsigned int y = 0xFF;    // FF hex = 1111 1111 bin = 255 dec
3 printf("%02X\n", x);      // Affiche "3A" (toujours 2 caractères par octet)

```

2 Représentation des Entiers

2.1 Entiers Non Signés (Unsigned)

Unsigned sur w bits

$$B2U_w(\vec{x}) = \sum_{i=0}^{w-1} x_i \cdot 2^i \quad \text{où } x_i \in \{0,1\} \text{ est le bit à la position } i.$$

Plage : $[0, 2^w - 1]$. Aucun nombre négatif représentable.

unsigned char : 11011011₂

1	1	0	1	1	0	1	1
2 ⁷	2 ⁶	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰

$= 128 + 64 + 16 + 8 + 2 + 1 = 219$

Type	Bits	Min	Max
unsigned char	8	0	255
unsigned short	16	0	65535
unsigned int	32	0	4294967295
unsigned long	64	0	$2^{64} - 1$

2.2 Complément à Deux (Two's Complement)

Two's complement sur w bits

$$B2T_w(\vec{x}) = -x_{w-1} \cdot 2^{w-1} + \sum_{i=0}^{w-2} x_i \cdot 2^i$$

Le bit de poids fort (MSB) x_{w-1} est le **bit de signe** : 0 = positif, 1 = négatif.

Plage : $[-2^{w-1}, 2^{w-1} - 1]$. **Asymétrie :** $|T_{min}| = |T_{max}| + 1$.

signed char: -3

1	1	1	1	1	1	0	1
---	---	---	---	---	---	---	---

signe
 $= -128 + 64 + 32 + 16 + 8 + 4 + 1 = -3$

signed char: +120

0	1	1	1	1	0	0	0
---	---	---	---	---	---	---	---

$= 0 + 64 + 32 + 16 + 8 + 0 + 0 + 0 = 120$

Valeurs remarquables (8 bits)

-1 : Tous les bits à 1 $\rightarrow 0xFF = 11111111_2$
 $T_{min} = -128$: Bit de signe à 1, reste à 0 $\rightarrow 0x80 = 10000000_2$
 $T_{max} = 127$: Bit de signe à 0, reste à 1 $\rightarrow 0x7F = 01111111_2$
Négation : $-x = \sim x + 1$ (inverser tous les bits, ajouter 1)

Négation : 5 $\rightarrow$ -5 (8 bits)

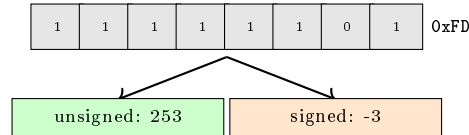
$x = 5$	0	0	0	0	1	0	1	0x05
$\sim x$	1	1	1	1	1	0	0	0xFA
$\sim x + 1$	1	1	1	1	1	0	1	0xFB = -5

3 Conversions Signed ↔ Unsigned

Principe de conversion

La conversion entre signed et unsigned **préserve le motif binaire** mais change son interprétation. Aucun bit n'est modifié.

Même motif binaire, deux interprétations



Formules de conversion (sur w bits)

$$\text{Signed} \rightarrow \text{Unsigned} : T2U_w(x) = \begin{cases} x & \text{si } x \geq 0 \\ x + 2^w & \text{si } x < 0 \end{cases}$$

$$\text{Unsigned} \rightarrow \text{Signed} : U2T_w(u) = \begin{cases} u & \text{si } u \leq T_{max} \\ u - 2^w & \text{si } u > T_{max} \end{cases}$$

Listing 2: Conversions en C

```
1 signed char sc = -1; // 0xFF en mémoire
2 unsigned char uc = (unsigned char)sc; // Toujours 0xFF, interprété comme 255
3 int x = -1;
4 unsigned int y = 0;
5 if (x < y) // FAUX! x converti en unsigned: 0xFFFFFFFF > 0
6     printf("x < y\n"); // N'est PAS exécuté!
```

Comparaisons signed/unsigned

Quand un opérande est signed et l'autre unsigned, le signed est **implicitement converti en unsigned**. Cela peut inverser le résultat attendu des comparaisons.

Pièges de comparaison (32 bits)

Expression	Type	Évaluation	Résultat
<code>-1 < 0U</code>	unsigned	<code>0xFFFFFFFF < 0</code>	0 (faux)
<code>-1 > -2</code>	signed	<code>-1 > -2</code>	1 (vrai)
<code>2147483647 > -2147483648</code>	signed	<code>T_{max} > T_{min}</code>	1 (vrai)
<code>2147483647U > -2147483648</code>	unsigned	<code>T_{max} > 2147483648</code>	0 (faux)

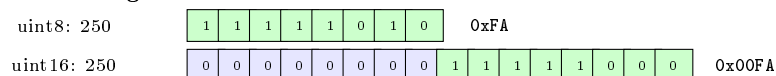
4 Extension de Signe

Extension lors de la promotion de type

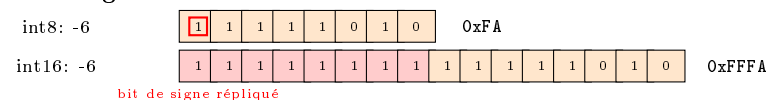
Quand on convertit un entier de w bits vers w' bits ($w' > w$) :

Unsigned : Extension par zéros (zero extension) **Signed** : Extension du bit de signe (sign extension)

Extension unsigned : 8 → 16 bits



Extension signed : 8 → 16 bits



Listing 3: Extension de signe en C

```

1 int8_t x = -6;           // 0xFA
2 int16_t y = x;          // 0xFFFA (bit de signe étendu)
3 uint8_t a = 250;        // 0xFA
4 uint16_t b = a;         // 0x00FA (zéros ajoutés)
5 int8_t neg = -1;        // 0xFF
6 uint32_t big = (uint8_t)neg; // 0x000000FF (255), pas 0xFFFFFFFF

```

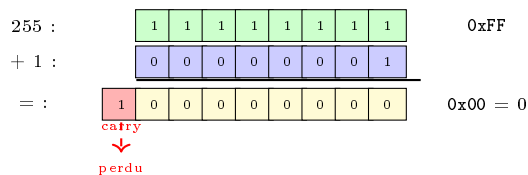
5 Dépassements et Débordements

5.1 Overflow Unsigned

Comportement unsigned (défini par la norme C)

L'arithmétique unsigned est définie **modulo** 2^w . Le débordement "wrap around" est **garanti**.
 Résultat = (valeur mathématique) mod 2^w

Wrap-around unsigned 8 bits : le carry perdu



Listing 4: Overflow unsigned

```

1 unsigned char x = 255;
2 x++;           // x = 0 (défini, wrap-around: 256 mod 256 = 0)
3 unsigned char y = 0;
4 y--;           // y = 255 (défini: -1 + 256 = 255)
5 // Détection d'overflow AVANT addition (sûr)
6 if (b > UINT_MAX - a) // Vérifie si a + b dépasserait UINT_MAX
7     printf("Overflow would occur!\n");

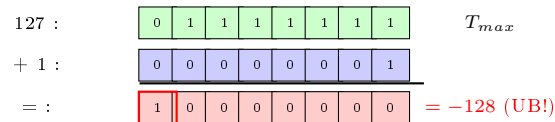
```

5.2 Overflow Signed

Comportement signed (Undefined Behavior)

Le débordement d'entiers signés est un **Undefined Behavior** (UB) en C. Le compilateur peut supposer qu'il ne se produit jamais et optimiser le code en conséquence. Le résultat est **imprévisible**.

Overflow signed 8 bits (UB) : changement de signe



Listing 5: Détection sûre d'overflow signed AVANT l'opération

```

1 #include <limits.h>
2 int a = 2000000000, b = 1000000000;
3 if ((b > 0 && a > INT_MAX - b) || (b < 0 && a < INT_MIN - b)) {
4     printf("Overflow se produirait!\n");
5 } else {
6     int sum = a + b; // Safe
7 }

```

Bugs de sécurité réels (CVE)

CVE-2009-1385 (Linux kernel e1000 driver): Un overflow d'entier permettait un buffer overflow.

CVE-2021-43267 (Linux kernel TIPC): Un overflow dans la validation de taille permettait une exécution de code.

6 Représentation IEEE 754 (Flottants)

6.1 Pourquoi les flottants ?

Les entiers ne suffisent pas pour : très grands nombres (1.5×10^{11} m), très petits nombres (9.11×10^{-31} kg), nombres fractionnaires (π , 19.99 €).

Le problème de la virgule fixe

Virgule fixe gaspille des bits : pour 0.000001 la partie entière est inutile, pour 10^{15} la partie fractionnaire est inutile. **Solution** : Faire "flotter" la virgule → virgule **flottante**.

6.2 Nombres binaires fractionnaires

Représentation binaire fractionnaire

$$b_m b_{m-1} \dots b_1 b_0 . b_{-1} b_{-2} \dots b_{-n} = \sum_{i=-n}^m b_i \times 2^i$$

Bits à gauche du point : poids $2^0, 2^1, 2^2, \dots$

Bits à droite du point : poids $2^{-1} = 0.5, 2^{-2} = 0.25, 2^{-3} = 0.125, \dots$

Représentation binaire fractionnaire : $101.1101_2 = 5.8125_{10}$

	Partie entière			Partie fractionnaire			
Bits:	1	0	1	1	1	0	1
Poids:	2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3}	2^{-4}
Valeur:	4	2	1	0.5	0.25	0.125	0.0625

$$\text{Valeur} = 4 + 1 + 0.5 + 0.25 + 0.0625$$

$$= 5.8125$$

Exemple : 101.11_2

$$101.11_2 = 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2} = 4 + 0 + 1 + 0.5 + 0.25 = 5.75$$

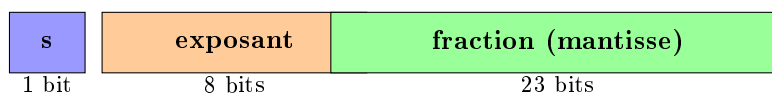
6.3 Formule IEEE 754

Formule IEEE 754

$$V = (-1)^s \times M \times 2^E$$

s = Signe (0 = positif, 1 = négatif), M = Mantisse (chiffres significatifs), E = Exposant (puissance de 2)

6.4 Structure d'un float (32 bits)



Décodage des champs

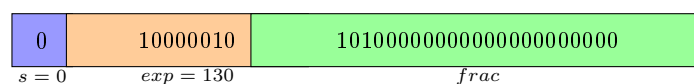
Signe : $s = 0 \rightarrow$ positif, $s = 1 \rightarrow$ négatif

Exposant biaisé : exposant stocké = $E + 127$ donc $E = \text{exposant stocké} - 127$

Mantisse avec "1 implicite" : $M = 1.\text{fraction}$ (le "1." n'est pas stocké)

6.5 Exemple : convertir 13.0 en float

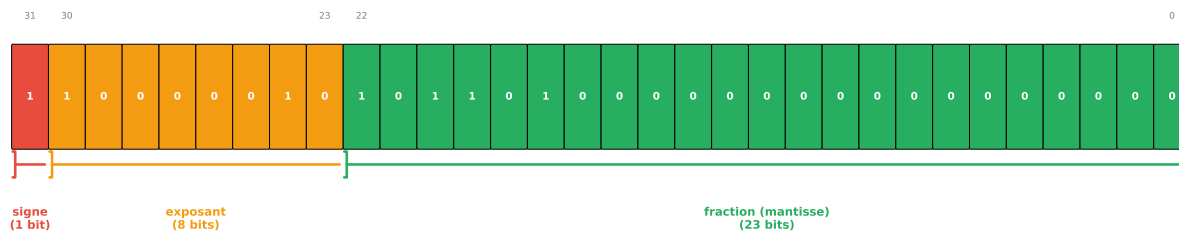
1. **Signe** : 13.0 positif $\rightarrow s = 0$
2. **Binaire** : 13 = 1101₂ donc 13.0 = 1101.0₂
3. **Normaliser** : 1101.0₂ = 1.101₂ $\times 2^3 \rightarrow E = 3$
4. **Exposant biaisé** : 3 + 127 = 130 = 10000010₂
5. **Fraction** : 101 + 20 zéros



Résultat : 0x41500000. **Vérification :** $(-1)^0 \times 1.101_2 \times 2^3 = 1.625 \times 8 = 13.0 \checkmark$

6.6 Exemple : -13.625

Structure IEEE 754 simple précision : $-13.625 = 0XC15A0000$



Décodage étape par étape :

1. Signe $s = 1 \rightarrow (-1)^1 = -1 \rightarrow$ négatif
2. Exposant $\text{exp} = 10000010_2 = 130_{10}$
 $E = \text{exp} - \text{Bias} = 130 - 127 = 3$
3. Fraction $\text{frac} = 0.101101000000000000000000_2$
 $M = 1 + \text{frac} = 1 + 0.7031250000 = 1.703125$

Formule IEEE 754 : $V = (-1)^s \times M \times 2^E$

Application : $V = (-1)^1 \times 1.703125 \times 2^3$
 $V = -1 \times 1.703125 \times 8$
 $V = -13.625 \quad \checkmark$

-  Signe
-  Exposant (biaisé)
-  Fraction (1 implicite)

Décortiquage de -13.625

1. Signe : négatif $\rightarrow s = 1$
 2. Binaire : $13 = 1101_2$, $0.625 = 0.101_2 \rightarrow 13.625 = 1101.101_2$
 3. Normalisation : $1.101101_2 \times 2^3$
 4. Exposant : $3 + 127 = 130 = 10000010_2$
 5. Fraction : $101101 + 17$ zéros
- Résultat** : 0xC15A0000

6.7 Valeurs spéciales

Valeur	Quand ?	Exemple en C
+0 et -0	Deux représentations de zéro	0.0f, -0.0f
$+\infty$ et $-\infty$	Dépassement de capacité	1.0f / 0.0f
NaN	Opération indéfinie	0.0f / 0.0f, sqrt(-1)

NaN : Not a Number

NaN n'est égal à rien, même pas à lui-même ! Utiliser `isnan(x)` pour tester.

```
float x = 0.0f / 0.0f; // x est NaN
if (x == x)           // FAUX! NaN != NaN
if (isnan(x))         // Correct
```

6.8 Les pièges des flottants

Piège n°1 : Représentation inexacte

$0.1_{10} = 0.0001100110011..._2$ (infini !). Le motif "0011" se répète $\rightarrow$ erreur d'arrondi.

Piège n°2 : Ne JAMAIS comparer avec

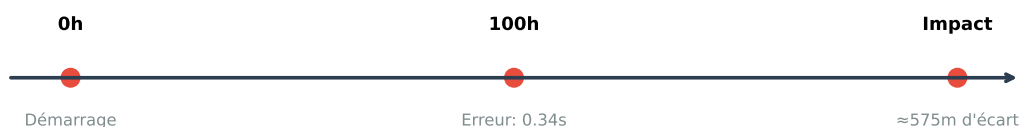
```
float a = 0.1f + 0.2f, b = 0.3f;
if (a == b) ... // DANGER! Peut être faux
if (fabsf(a - b) < 0.00001f) // Correct: comparer avec tolérance
```

Piège n°3 : Addition non associative

```
float a = (3.14f + 1e10f) - 1e10f; // = 0.0 !
float b = 3.14f + (1e10f - 1e10f); // = 3.14
```

6.9 Cas réel : Patriot Missile (1991)

Tragédie du Patriot (25 février 1991)



0.1 seconde $\approx 0.0001100110011..._2$ (infini) $\rightarrow$ Erreur/tick: 95.37 ns $\rightarrow$ Après 100h: 0.34s $\rightarrow$ À 1676 m/s: 575m

28 soldats américains tués — Le Scud n'a pas été intercepté

Ce qui s'est passé

Date : 25 février 1991, Guerre du Golfe, Dhahran, Arabie Saoudite.

Bug : L'horloge comptait en incréments de 0.1 s approximé sur 24 bits.

Après 100 heures : Erreur cumulée ≈ 0.34 s. À 1676 m/s $\rightarrow$ erreur de ≈ 570 m.

Conséquence : Le Scud n'a pas été intercepté. **28 soldats tués.**

6.10 Application en Intelligence Artificielle

Problème du Softmax en Deep Learning

$$\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

[X] Problème (naïf)

$x = [1000, 1, 2]$

$e^{1000} = +\infty$ (overflow)

Résultat: NaN

soustraire
 $\max(x)$

[OK] Solution (stable)

$x - \max(x) = [0, -999, -998]$

$e^0 = 1, e^{-999} \approx 0, e^{-998} \approx 0$

Résultat: [1.00, 0.00e+00, 0.00e+00]

Le problème du softmax

$\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_j e^{x_j}}$ est utilisé partout en IA.

Problème : $e^{1000} = +\infty$ (overflow), $e^{-1000} = 0$ (underflow) $\rightarrow$ NaN

Solution : Soustraire le maximum avant de calculer (numériquement stable).

Formats modernes pour l'IA

Format	Bits	Usage
float32	32	Standard, haute précision
float16	16	Entraînement GPU accéléré
bfloat16	16	Google TPU
int8	8	Inférence quantifiée

7 Bonnes Pratiques

7.1 Types portables

Utiliser les types de taille fixe (<stdint.h>)

Type signé	Type non signé	Taille
int8_t	uint8_t	8 bits
int16_t	uint16_t	16 bits
int32_t	uint32_t	32 bits
int64_t	uint64_t	64 bits

Listing 6: Types portables

```
1 #include <stdint.h>
2 #include <inttypes.h>
3 uint32_t counter = 0;
4 printf("counter = %" PRIu32 "\n", counter);
```

7.2 Éviter les pièges signed/unsigned

Listing 7: Comparaisons sûres

```
1 int index = -1;
2 unsigned int limit = 100;
3 if (index >= 0 && (unsigned int)index < limit) // BON: cast explicite
4     printf("Index valide\n");
```

7.3 Options de compilation recommandées

Listing 8: Options gcc

```
1 gcc -Wall -Wextra -Werror -Wsign-compare -fsanitize=undefined -g prog.c
```